A Pipeline Toward an Automated Conjecturing-Refuting-Formalizing-Proving Loop for Graph Theory

Ján Pastorek[1,*]

[1]Department of Applied Informatics, Comenius University in Bratislava, Bratislava, Slovakia

Abstract

We report on our ongoing project to develop a computational pipeline for an automated graph-theoretic conjecturing-refuting-formalizing-proving system. Generation is counterexample-guided: a Graffiti3 generator (the native nonlinear engine of the TxGraffiti package) proposes inequalities over a small seed of a few hundred graphs that grows only by hard counterexamples. We add four components that, together, alter the nature of the output: (i) a refutation dataset of ∼348,000 graphs unioning the complete House of Graphs invariant export, the exhaustive census of all connected graphs on at most nine vertices, and several special extremal graph families (strongly regular, minimal Ramsey, Cayley, cages, barbells, lollipops, spiders, etc.), augmented by on-demand class-aware random models (random-regular, $G(n, p)$, random trees and random bipartite graphs), against which every candidate conjecture is attacked; (ii) a *novelty filter* of 203 classical theorems known from literature and trivial theorems closed under transitive composition and linear identity substitution, which decides whether a candidate is implied by them; (iii) a *counterexample search stage* to find counterexamples to surviving conjectures; and (iv) an *autoformalization stage* that translates surviving conjectures into Lean 4 [1] statement skeletons, every candidate proof kernel-verified against a pinned `mathlib4` and our custom invariant preamble. On the cluster we attempt the simplest machine-generated survivors with two state-of-the-art neural provers—DeepSeek-Prover-V2-671B (served with vLLM) and the Lean-specialised OProver-32B—but in single-shot whole-proof mode neither verifies any survivor: the conjectures are expressed over *custom* graph invariants outside the provers' training distribution, and closing the conjecture→proof loop end-to-end remains open. Multi-round agentic proving with compiler feedback and invariant-definition grounding is under evaluation.

Keywords

automated conjecturing, automated theorem proving, graph theory, counterexample search, TxGraffiti

1. Introduction

Computer-assisted discovery of mathematical results and conjectures has a long history and with the advent of artificial intelligence and high-performance computing, it is increasingly recognized as valuable in the mathematical community.

Computer-assistance in this area can be roughly classified into four corresponding areas based on the research stage it assists: *conjecturing* new statements based on a finite database, *refuting* them by searching for counterexamples, *formalizing* them into a rigorous language, and ultimately *proving* them in a formal system.

Automated conjecturing in graph theory has a forty-year history, from Fajtlowicz's Graffiti [2, 3, 4] and DeLaViña's Graffiti.pc [5, 6] to the AutoGraphiX/GraPHedron programme of Aouchiche, Caporossi, Hansen, and Mélot [7, 8, 9] and, more recently, Davila's TxGraffiti [10, 11] and The Optimist [12], Larson's Dalmatian-based Conjecturing [13, 14] and Graffiti^3 [15], and the polyhedral PHOEG system [16]. These systems propose inequalities between graph invariants—$\alpha(G) \leq \nu(G)$, $\chi(G) \leq \Delta(G) + 1$, and so on—by fitting candidate bounds to a finite database of graphs and retaining and sorting only those that pass a set of filters and heuristics. Many resulting conjectures have become theorems; some have been refuted by hand or by machine search, see for instance [17, 14, 18].

ITAT 2026: Information Technologies – Applications and Theory, September 25–29, 2026, Vršatec, Slovakia
email: jan.pastorek@fmph.uniba.sk (J. Pastorek)
orcid: 0000-0001-8237-1275 (J. Pastorek)


A practitioner who reimplements such a system quickly encounters five possible failure modes. First, a candidate can hold on every graph in a finite database yet be *false in general*: the database simply lacks the structure that breaks it. Second, the system can produce a candidate that is *true but known*, already recorded in the literature. Third, the system can produce a candidate that is *true but trivial*, resulting in an "avalanche of triviality". Fourth is the *monster conjecture*: a single conjecture that overfits the table by accumulating many terms and constants (producing unnecessarily complex bounds such as $\alpha \leq 0.31\,\Delta + 0.42\,\nu - 0.07\,m + 1.8$). Fifth, the system can "discover" a counterexample to a well-established conjecture, which almost always signals an error in an invariant routine or a misread statement rather than a result. The first and the last failure modes are about *trust*: a generated inequality that "holds" and a verified "counterexample" are each only as reliable as the data and invariant computations behind them. The second and the third failure modes are about *novelty*: a candidate that is true but known, or true but trivial, is mathematically uninteresting and of little value to the practitioner. All of these failure modes are addressed in our pipeline by the adversarial counterexample search, the novelty filter, and the selection heuristics.

To address the first and fourth failure modes—false bounds and overfitting—a robust refutation engine is required. Computational methods used for searching for (counter)examples of graphs have been recently summarised in a review article of Jorik Jooken [18]. Among others, the methods include exhaustive search with branch and bound, randomised search, and heuristic search. More recently, deep reinforcement learning has been applied to refute several conjectures [19].

Lastly, there is a growing body of recent work on autoformalization and automated theorem proving especially in Lean, including for graph theory, see for instance [20, 21, 22, 23, 24, 25]. However, although we recently found in [15] certain indications in this direction, to the best of our knowledge there is no existing system that integrates automated conjecturing, refuting, formalizing, and proving in a closed loop for graph theory.

In this paper, we describe and report on the development of such a computational system AutoGraphForge that aims to combine (i)-(iv) in a pipeline. The first two stages are often combined into a *generate-refute* loop, where conjectures are generated and then tested against a database of known graphs and invariants, with counterexamples being added to the database for future iterations. The last two stages are often combined into a *formalize-prove* loop, where conjectures are translated into a formal language Lean and then attempted to be proved using automated theorem provers.

Our current contributions are:

1. We build the first version of the computational pipeline AutoGraphForge and run it on a high-performance computing cluster: a multi-round counterexample-guided inductive synthesis (generate-refute) loop partitioned into five full-node SLURM jobs (each partitioning the invariant list), followed by a merge and a GPU proving stage that serves DeepSeek-Prover-V2-671B with vLLM to kernel-verify the simplest survivors. The end-to-end proving results of this scaled run are still in progress; the proving results reported in this paper are those of the in-process 7B model (§3.5).
2. We build a refutation dataset of 348,207 graphs (carrying up to 59 exactly-computed invariants), combining the complete House of Graphs (HoG) [26] export and many other extremal graph families, including the strongly-regular, minimal-Ramsey, Cayley, cage, cograph, and minimally-rigid families, barbells, lollipops, spiders, and complements of triangle-free graphs, together with on-demand class-aware random models. Generation itself runs over a small evolving seed (a few hundred graphs), not the full dataset.
3. A novelty filter of 203 classical or trivial theorems (§3.4), closed under transitive composition of $f \leq g$ relations and under linear identity substitution (e.g. the regular-graph identity $\Delta = \delta = 2m/n = \rho =$ degeneracy, König's $\nu = \tau$, Gallai's $n = \alpha + \tau$), implemented as a small linear-program feasibility test. The filter is a *sound but incomplete* implication test: a positive verdict certifies that a candidate is genuinely implied by the tabled theorems and identities (not merely that it matches a template), while a candidate that is in fact derivable through inter-invariant relations not encoded in the table may still pass through as "novel."

4. A permanent *adversarial verification stage* (§3.5.1): every candidate is tested against on-demand structure-targeted datasets (§3.2)—a parametric/atlas dataset (barbells, lollipops, spiders, class generators, complements of triangle-free graphs) and a class-aware random-models dataset—constructed to break loose bounds; in the single-pass run reported in §3.5.1 these materialised 1,142 and 219 graphs respectively. This stage runs first in the falsification loop and stores found counterexamples back into the dataset.
5. A validated counterexample-search engine (§4.2) that includes among others recent state-of-the-art methods such as probabilistic search using the cross-entropy method and Monte Carlo methods. To validate it we reproduced the refutation of a number of conjectures and exhaustively verified a range of open conjectures (§4.2).

We note that our system is not built from scratch but is built upon existing packages, frameworks and methodologies present in the state-of-the-art literature on each of the three areas of conjecturing, refuting, and formalizing and proving. In particular, the conjecture generation is built on top of TxGraffiti; the counterexample search engine is built upon recent state-of-the-art methods such as probabilistic search using the cross-entropy method and Monte Carlo methods described in [18]; and the formalization and proving layer is built upon recent autoformalization methods and automated theorem provers, specifically Lean 4 with `mathlib4` as the target language and the DeepSeek-Prover-V2 automated prover [25].

2. Preliminaries

All graphs in this paper are considered finite and simple. For a graph G we write $n = n(G)$ for its *order* (number of vertices) and $m = m(G)$ for its *size* (number of edges). We use standard invariant symbols throughout: maximum and minimum degree Δ, δ, average degree $2m/n$, and *degeneracy*; independence number α, matching number ν, vertex-cover number τ, clique number ω, chromatic number χ, chromatic index χ', domination number γ, and independent-domination number i; vertex- and edge-connectivity κ and λ; radius rad, diameter diam, and triangle count tri; and the largest adjacency eigenvalue, or *spectral radius*, written λ_1 and equivalently ρ ($\lambda_1 = \rho$; neither to be confused with the edge-connectivity λ). The *barbell* $K_a - K_b$ denotes two disjoint cliques joined by a bridge (a standard, informal notation). On a regular graph ρ equals the common degree, giving the identity $\Delta = \delta = 2m/n = \rho$. The clique-cover number—the minimum number of cliques covering $V(G)$, equivalently $\chi(\overline{G})$—is written $\overline{\theta}$ throughout.

3. The AutoGraphForge Architecture

3.1. System overview

At a high level, AutoGraphForge is a continuous, self-critical loop (Figure 1) that couples generation, selection heuristics, refutation search, and a formalization/proving layer, in the spirit of the competing optimist/pessimist "GraphMind" architecture of The Optimist [12]. Candidate conjectures are scored and ranked, then aggressively tested by the refutation engine (the "pessimist"); only statements that survive multiple rounds advance to autoformalization, proving, and human review, while every counterexample feeds back into the knowledge base. The remaining sections detail each stage in turn: the data (§3.2), generation and selection (§3.3), refutation (§3.5), and formalization and proving (§3.6).

3.2. Invariants and the database

Trust in generated inequalities is bounded by trust in the data. We combine several sources. The HoG export [27, 26] provides 28,859 graphs with ~28 pre-computed invariants each (treewidth, feedback-vertex-set number, spectral data, vertex cover, girth, ...), parsed directly so that nothing is recomputed; values marked `undefined`, `infinity`, or `Computation time out` are treated as missing. Database also contains every connected graph on $2 \le n \le 9$ vertices (273,192 graphs) through an exact invariant list

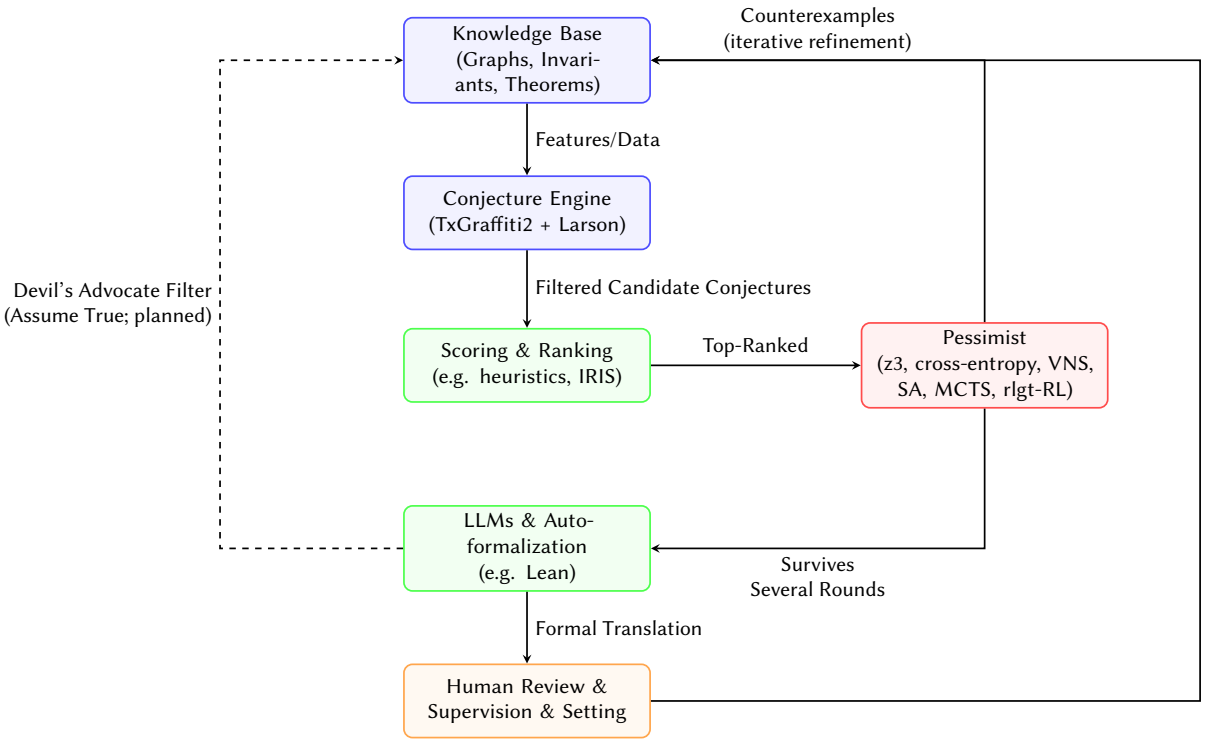

Figure 1: The integrated automated mathematical discovery pipeline. Conjectures are generated, immediately scored and ranked (heuristics, IRIS), and then aggressively tested by the refutation engine (the Pessimist: adversarial dataset plus z3/cross-entropy/VNS/SA/MCTS/rlgt search). Only conjectures surviving multiple rounds of refutation are passed to LLMs for autoformalization; found counterexamples are fed back into the knowledge base. The dashed line is the planned Devil's Advocate filter (future work; see the Conclusion), where surviving conjectures would be assumed true to constrain future generation, before advancing to final human review.

and a set of computed graph-class flags (triangle-free, cubic, claw-free, cograph, split, outerplanar, self-complementary, Hamiltonian, well-covered). We also used the HoG meta-directory to collect cograph, minimal-Cayley, cage, minimal-Ramsey, strongly-regular (Spence), and minimally-rigid graph families. The union contains 348,207 graphs and up to 59 exactly-computed invariant columns. This static dataset is one refutation resource; alongside it the refuter draws on two further datasets computed on demand with the exact list: parametric families (barbells, lollipops, spiders, k-trees, line graphs, Delaunay graphs, $\alpha = 2$ complements, ...) and random graph models (random-regular, Erdős–Rényi $G(n, p)$, random trees, and random bipartite graphs, several per (class, order)), so that refutation probes the average invariants corresponding to each class and not only structured or special witnesses. Conjecture *generation*, by contrast, runs over a small initial seed described in §3.3.1.

Backend invariant calculation. A single backend, GraphCalc [28], is designated *canonical*: every value that enters the store is its exact computation, or left empty. An independent `networkx` [29] list is registered purely as a cross-check, and a `crossvalidate` script compares the two on sample graphs before any build is trusted—reproducing the zero-mismatch result below, now as an automated precondition rather than a one-off audit. The two connectivity invariants GraphCalc lacks (κ, λ) are sourced from `networkx` and likewise blanked when unavailable; no value is ever silently approximated by a second backend.

Validation against an authoritative reference. We verified computation of invariants of GraphCalc against the independently-computed HoG values on 2,500 graphs: for every shared invariant ($\chi, \alpha, \omega, \gamma, \nu, \Delta, \delta, \kappa, \lambda$, diam, tri, and the booleans) there were *zero* mismatches.

3.3. Conjecture generation as theory selection

Treat the examples and counterexamples as rows of a common invariant table T, whose numerical columns embed the objects as a point cloud in $\mathbb{R}^k$ (where k is the number of invariants)—the *invariant cloud*. A linear grammar of conjectures consists of halfspaces $\sum_i a_i f_i(G) \le c$ (with $a_i, c \in \mathbb{R}$) that must hold for every row of the current snapshot. The convex hull of the cloud makes the core difficulty geometric: there are infinitely many snapshot-true inequalities (any halfspace whose boundary lies above the hull), but only the *facets*—tight on at least one extremal object—carry information, and no single inequality is "the best." A practical system is therefore not fitting one predictor but choosing a *budgeted, nonredundant* theory: a small library of cuts that each strengthen the current envelope somewhere. Our generation and selection layers (§3.3) implement exactly this budget.

This builds on three distinct paradigms that have emerged for searching the space of graph-theoretic inequalities. *Algebraic expression trees*, used by GRAFFITI, build candidate conjectures as rooted trees of invariants combined by unary and binary operations (sums, products, square roots), and enumerate candidates by exhaustive or heuristic tree search. *Linear and mixed-integer programming*, used by TXGRAFFITI [10, 11], THE OPTIMIST [12], and GRAFFITI^3 [15], instead solves an optimisation model over a precomputed tabular dataset: by minimising the distance between a target invariant and a linear combination of others, the tightest possible bound is found without enumeration. The *polyhedral/geometric* approach of GRAPHEDRON [9, 8] and PHOEG [16, 30] embeds each graph as a point in invariant space and reads off conjectures as facets of the resulting convex hull; this produces the *complete* set of optimal linear inequalities for those invariants and makes optimality a geometric certificate.

The methods are released as a single package, AutoGraphForge, that takes TXGRAFFITI2 [10] as its foundation. The canonical representation of a conjecture is TxGraffiti2's own `Conjecture` object, and every subsequent component—the implication-deciding novelty filter, the selection heuristics, and the refutation engine—is built to take these native objects directly.

3.3.1. Architecture and linear generation

We build upon TxGraffiti2's native convex-hull and ratio generators and its implementation of the Dalmatian dominance filter, meaning the generation logic operates directly on Pareto-optimal frontiers of non-dominated, tight bounds without redefining them. Invariants are supplied by GRAPHCALC (§3.2). In the runs reported here the generator is GRAFFITI3 [15] (the native nonlinear conjecturing engine of the TxGraffiti package) which additionally mines ratio, product, $\sqrt{\cdot}$ and log bounds together with *Sophie* sufficient-conditions over the seed's 59-invariant GRAPHCALC list.

Generation begins from the TxGraffiti *expressive graphs* (a curated default dataset) and grows only by hard witnesses. The single-round figures quoted in §4.1 come from an earlier one-pass run; the HPC run (§4.1) instead iterates for up to 25 generate–refute rounds, stopping early if it cannot find counterexamples to any conjectures or at a wall-clock budget, so that each round conjectures over a strictly larger, counterexample-hardened seed.

3.3.2. Product and ratio conjectures

Ratios reduce to linear bounds ($f/g \le c \iff f \le cg$), so the genuinely new expressive power is in products. The product sweep rediscovered the classical product theorems—$n \le \alpha\chi$ (Gallai [31]), $n \le \gamma(\Delta+1)$ and $n \le \alpha(\Delta+1)$ (greedy/domination)—which is precisely the desired correctness signal, together with provable handshake bounds such as $\alpha\delta \le m$ (the edges incident to an independent set number at least $\alpha\delta$). After adversarial filtering, the product remainder is, like the linear remainder, dominated by known or provable bounds.

3.4. Novelty filtering and dominance heuristics

Without aggressive filtering, conjecture systems suffer a "combinatorial explosion," generating thousands of trivial or redundant statements. *Simplicity Over Complexity*: While optimization models can handle

bounds with many invariants simultaneously, based on an Ockham's razor principle, bounds with both fewer invariants, and arithmetic and logical operations are preferred. *Strongest-Conjecture principle* requires that the system assert only the tightest bound for which no counterexample is known; this is the formal basis of the Dalmatian dominance filter and the polyhedral facet approach alike. The *Dalmatian heuristic* (dominance), due to Fajtlowicz and revisited by Larson and Van Cleemput [2, 13], discards a candidate whenever some accepted conjecture is never weaker and sometimes strictly tighter; this is the primary filter used throughout our pipeline (§3.3), alongside a linear programming novelty filter that decides whether an inequality is a trivial combination of already known mathematical relations (§3.4). We also utilise the *Sophie heuristic*, which shifts from producing a numeric bound to identifying a Boolean property that fully characterises all sharp graphs, converting an inequality into a structural characterisation.

Discovering a candidate inequality is only part of the task: the system must also decide whether the inequality is genuinely new or merely a combination of already-known relations.

Suppose the system generates a candidate inequality

$$f \leq R(g),$$

where f is a target invariant (for example, the independence number) and $R(g) \triangleq \sum_i c_i g_i + c_0$ is a linear combination of other invariants g_i, with $c_i, c_0 \in \mathbb{R}$. Before accepting it, the system tests it against a curated library of known theorems that bound the same target invariant ($f \leq B_1(g)$, $f \leq B_2(g)$, and so on, where each $B_j(g)$ is a known affine bound on f).

The candidate is declared *known* (redundant) if it is implied by a convex combination of these theorems. Assigning a non-negative weight w_j to each $B_j(g)$ with $\sum_j w_j = 1$ yields the aggregate bound $f \leq \sum_j w_j B_j(g)$. If this aggregate is at least as tight as $R(g)$, the candidate is redundant; equivalently, the difference between the candidate and the aggregate is non-negative everywhere:

$$R(g) - \sum_j w_j B_j(g) \geq 0.$$

The non-negativity certificate

To certify that this difference is non-negative, we express it identically as a sum of terms that are individually non-negative. We seek weights $w_j \geq 0$, non-negative slack variables $s_i, t \geq 0$, and free multipliers $\mu_k \in \mathbb{R}$ such that the following identity holds for all g:

$$R(g) - \sum_j w_j B_j(g) = \sum_i s_i (g_i - L_i) + \sum_k \mu_k E_k(g) + t.$$

The right-hand side is composed of terms each guaranteed to be non-negative:

- *Safe lower bounds* $s_i(g_i - L_i)$: every invariant g_i has a known minimum value L_i, so $(g_i - L_i) \geq 0$; multiplying by a non-negative slack s_i preserves non-negativity.
- *Class identities* $\mu_k E_k(g)$: structural relations $E_k(g) = 0$ that hold identically on a graph class P_k, so multiplying by any free multiplier μ_k does not affect non-negativity. Crucially, soundness requires that E_k strictly vanish on the whole domain the candidate is quantified over, so each identity is *class-gated*: it is admitted only when the candidate's hypothesis class is P_k or a subclass of it. For an *unconditioned* candidate, only *universal* identities (those with P_k = all graphs, e.g. Gallai's $n = \alpha + \tau$) are available; a class-restricted identity such as König's $\nu = \tau$ (bipartite only) is never used to declare an unconditioned candidate known, which would otherwise be an unsound false positive.
- *Constant slack* t: a non-negative scalar absorbing the residual constant.

Because the invariants g_i are treated as independent symbolic variables, matching the coefficient of each g_i on both sides reduces the problem to a system of linear equations. Verifying redundancy therefore reduces to a linear-programming (LP) feasibility test: if the solver finds weights and slacks

satisfying the identity, the candidate is provably a structural consequence of the tabled theorems and is discarded.

The table holds 203 entries after closing the simple $f \leq g$ relations under transitivity; the class-gated identity multipliers μ_k capture, in one mechanism, Whitney's connectivity chain $\kappa \leq \lambda \leq \delta$ [32] (all graphs), König's $\nu = \tau$ [33] (bipartite), and Gallai's $n = \alpha + \tau$ [31] (all graphs)—each applied only on the class where it holds. The filter correctly hides, for example, $\kappa \leq \frac{1}{2}\delta + \frac{1}{2}\lambda$.

3.4.1. Sophie layer: sufficient conditions

Using the *Sophie* heuristic (introduced in §3.4) directly from its GRAFFITI[3] implementation, the same machinery, in its boolean mode, produces *sufficient conditions* for a graph property rather than numeric bounds: statements $P_1 \wedge \cdots \wedge P_k \Rightarrow Q$ (where $k \geq 1$) built from a set of boolean predicates (regularity, planarity, claw-freeness, chordality, 2- and 3-connectivity, the Dirac threshold $2\delta \geq n$, vertex transitivity, even order, ...) with the logical operators $\neg, \wedge, \vee, \rightarrow$. Targeting Hamiltonicity and the existence of a perfect matching, with Dirac's theorem [34] supplied as the theory and the same adversarial filter applied, the search recovers genuine theorems—most strikingly Sumner's result [35] that a connected claw-free graph of even order has a perfect matching (even order $\wedge$ claw-free $\Rightarrow$ perfect matching)—alongside plausible conditions such as 2-connected $\wedge$ claw-free $\Rightarrow$ Hamiltonian and 3-connected $\wedge$ planar $\Rightarrow$ Hamiltonian. That a system rediscovers Sumner's theorem from data, rather than fitting an artefact, is again the desired correctness signal. In the HPC run these sufficient-conditions are no longer merely validated at generation time: each hyp $\Rightarrow$ property is routed through the same tiered refutation dataset as the inequalities, refuted by any graph on which the hypothesis relation holds but the property fails (NaN-aware, so a dataset lacking the property is a coverage miss, never a false refutation). This is what removes spurious sufficient-conditions such as (order ≤ 14) $\Rightarrow$ connected that hold across a connected-only snapshot but are false in general.

3.5. Iterative refutation and adversarial search

Because the target class is infinite and the snapshot is a biased partial view, "true on the table" is a statement about present *evidence*, not a generalisation claim. The proper analogue of a test set is therefore not a static validation set but an *iterative counterexample search*: an actively targeted search for objects outside the table on which proposed laws fail. When this search succeeds, the counterexample is promoted into the table, changing the feasible space of all subsequent conjectures. This makes conjecturing intrinsically online—the dynamic-database principle—and it is realised concretely by the adversarial dataset, the counterexample-search backends, and dynamic re-insertion of found counterexamples (§3.5). Automated discovery depends on this adversarial loop. The refutation rules of the GRAFFITI tradition—the *Red Burton rule* (minimising counterexamples), the *TicTac method*, and *Pepper's rule* (treating every machine-generated conjecture as false until counterexamples are exhausted)—were documented by DeLaViña [6]. More recently, Monte-Carlo algorithms [36, 37] and probabilistic optimization [19] have been applied directly to automated refutation (for a review see [18]), building counterexample graphs edge by edge guided by the conjecture's violation margin.

Generation is inexpensive; rejection is the decisive step. Every candidate that survives novelty and selection is passed to the adversarial stage.

3.5.1. Adversarial verification

A finite database cannot certify a universally-quantified statement. We make this concrete: the bound $\delta(G) \leq \lambda(G) + 5$ holds on all 347,855 graphs of the combined dataset on which λ is defined, yet is false—the barbell $K_{10} - K_{10}$ has $\delta = 9, \lambda = 1$. The combined dataset simply contains no graph with $\delta - \lambda > 5$.

The adversarial stage therefore tests each candidate against structure-targeted datasets with exact invariants—generated on demand (§3.2), materialising 1,142 parametric/atlas graphs (families) and 219 class-aware random graphs for the single-pass run quoted below—of barbells (large δ, small λ), cliques

with sparse tails (high χ/ω, low average degree), spiders (large radius, small γ), random regular graphs, k-trees, line graphs, Delaunay (planar) graphs, and complements of triangle-free graphs ($\alpha = 2$). The linear artefacts $\delta \leq \lambda + 5$ and $\chi \leq 2m/n + 3$ are typically refuted instantly by this collection. The stage runs first in the falsification loop and stores any counterexample it finds in a growing dataset, so refutations accumulate across rounds.

Dynamic database. Any found counterexample is inserted into the database, iteratively tightening subsequent candidate conjectures in the next rounds. This realises the iterative counterexample search loop of §3.5 in code: the feasible region of invariant space contracts with every refutation, and the same artefact is never proposed twice.

3.5.2. Counterexample-search backends

For conjectures whose counterexamples lie outside any fixed dataset, the system provides four interchangeable search backends, all sharing a single "margin$(G) > 0 \iff G$ refutes" interface and tried cheapest-first per candidate:

- *SMT (satisfiability-modulo-theories) encoding* (z3): graph structure is encoded as Boolean edge variables; invariant constraints (k-colorability, independence set size, etc.) are expressed as pseudo-Boolean formulae and dispatched to a Z3 [38] solver; most effective for graphs on up to ~ 10 vertices.
- *Linear cross-entropy* (cross_entropy): Wagner's cross-entropy method [19] without a neural network — a per-edge inclusion distribution is sampled in batches, the elite fraction with the largest margin is retained, and the distribution is moved toward the elite edge frequencies. Gradient-free and cheap.
- *Variable-neighbourhood search* (vns): cycles through increasingly large flip neighbourhoods (1-edge → k-edge → vertex rewiring), accepting any slack-reducing move and restarting from a structured seed family (§4.2) when trapped in a local minimum.
- *Monte-Carlo tree search* (mcts): a standard UCT bandit search over edge modifications, guided by the conjecture's negative slack as the rollout reward.
- *Simulated annealing* (sa): standard simulated annealing over edge modifications, adopting a temperature schedule to escape local optima.
- *Deep reinforcement learning* (rlgt-RL): building on RLGT [19], trains an edge-selection policy guided by the conjecture's negative slack as reward.

All but z3 are black-box over the GraphCalc list. A hard per-phase wall-clock cap bounds the search: because an exact ILP invariant (χ, ω, α) on a large search graph runs in a C extension that a Python timer cannot interrupt, the parallel process pool is run under a deadline and any still-running evaluation is killed (its candidate kept as a survivor, never a false refutation).

Structure-seeded search. The backends above are driven from a structured seed library rather than from scratch: a large library of parametric graphs (barbells, double stars, lollipops, split graphs, spiders, regular graphs, $\alpha = 2$ complements) is generated for each order n, and each seed is driven to a local optimum of the violation margin by best-improvement edge-flip hill climbing (with degree-preserving 2-swaps for regular-graph targets). Plain cross-entropy over independent edge probabilities was tried first and *failed* its validation—it could not represent the block structure of a barbell—which is why seeding from structured graphs is essential. In the HPC run the search is swept over a wide order set ($n \in \{6, 7, \ldots, 12, 14, 16, \ldots, 50, 60, 75, 100\}$), with the per-order trial count scaled down as the order grows but floored so that even the largest orders receive a meaningful number of attempts, and with a per-candidate wall-clock budget that bounds round time regardless of how expensive the exact invariants become at large n; this lets the engine search for counterexamples beyond the exhaustive-census range while keeping each round time-bounded.

3.6. Formalization and proving

Surviving conjectures that pass refutation are handed to the formalization and proving layer. They are autoformalized [20] into Lean 4 [1] statement skeletons by a two-stage LLM pass (decompose the informal statement into sub-goals, then map each to `mathlib4` definitions); the output requires subsequent elaboration by the Lean kernel and is not a certified proof. Because of its size, DeepSeek-Prover-V2-671B is served with vLLM [39] in tensor-parallel across eight H200 GPUs; the Lean-specialised OProver-32B [40] (a Qwen3-based whole-proof prover, state-of-the-art on MiniF2F among open-weight provers) is served on a second node. Each of the 43 survivors that autoformalize to a type-correct goal is attempted with pass@k sampling, and every candidate is kernel-verified *independently* against `mathlib4` [41] library and our `GraphInvariants` preamble through a bare-`lean` `LEAN_PATH` check (no network, no mutation of the package tree). In single-shot whole-proof mode *neither prover verifies any survivor* (0/43 each); the completed generations fail with genuine Lean errors—failed tactics (e.g. `simp` making no progress, `rfl` on a non-definitional equality), unsolved goals, and instance-resolution failures—rather than timeouts or harness errors. We attribute this primarily to distribution shift: the survivors are stated over *custom* graph invariants (`dominationNumber`, `slaterNumber`, the zero-forcing family, ...) defined only in our preamble and absent both from `mathlib4` and from the provers' training corpora, so the model cannot lean on a known lemma library. We are additionally evaluating OProver in its native agentic mode—multi-round compiler-feedback repair with the invariant definitions injected as retrieval grounding—which is how it attains its reported benchmark scores. As of writing no machine-*generated* survivor has been proved end-to-end, so the conjecture→proof loop is not yet closed; we regard this as an honest measurement of the gap between current neural provers and out-of-distribution, custom-invariant graph conjectures, not as evidence the conjectures are false—they survived the full refutation pipeline.

4. Discovery in graph theory

We now report the pipeline's discoveries in graph theory, separating the Iterative run dynamics, a validated search engine, exhaustive checks of open conjectures, and a detailed account of the false positives.

4.1. Iterative run dynamics on the cluster

To establish a baseline, an initial single-pass generation run (one generate–refute round) yielded 3,959 candidates, of which 1,249 were refuted. The stratified exact datasets dominated the rejections (865 by the family dataset, 217 by the random models dataset, and 161 by the House of Graphs dataset), validating the strategy of testing against fast, parametrised datasets before resorting to expensive active search.

Building on this baseline, we ran AutoGraphForge as a SLURM array of five independent partitions on a high-performance computing cluster.[1] The total computational cost of the experiment amounted to 1.22 CPU years and 117 GPU hours.

Each partition (a full 256-core CPU node) owned a contiguous block of nine of the 45 numeric target invariants, all starting from a common seed of 2,860 graphs (the TxGraffiti expressive graphs plus accumulated hard witnesses, each carrying the exact 59-invariant GraphCalc list) and each running the generate–refute loop to a fixed point or a 29 h wall-clock cap. Table 1 summarises all five partitions.

Three patterns are robust across partitions. *(i)* The first round alone contributes ∼500–790 new hard witnesses (a ∼20–28% seed expansion); subsequent rounds decline sharply (see Table 1 for details), and the loop reaches a fixed point—a round that adds zero witnesses, leaving the seed and hence the next

[1] CPU workloads were deployed on HPE Cray XD2000 nodes (2× AMD EPYC 9745, 256 cores, 1,536 GB RAM, 200 Gb/s NDR200 InfiniBand). GPU formalization workloads ran on the PERUN AI accelerated partition containing HPE ProLiant Compute XD685 nodes (2× AMD EPYC 9535, 128 cores, 8× NVIDIA H200 141 GB GPUs, 2,304 GB RAM, 400 Gb/s NDR InfiniBand with 900 GB/s GPU-to-GPU bandwidth).

Table 1
Per-partition iterative dynamics (all five partitions). Witnesses are new hard counterexamples added back into the seed; a round adding zero witnesses is a fixed point.

Partition (target group)	Rounds	R1 witns.	Survivors	Seed
0 (χ, ω, clique, burning, dist.)	5	+793	1,762	2860 → 3850
1 (α, ν, γ, dom. family)	9	+499	1,438	2860 → 3475
2 (order, degrees, dom. variants)	5	+494	1,815	2860 → 3403
3 (size, slater, spectral, roman dom.)	5	+562	1,345	2860 → 3505
4 (forcing nums, τ, triameter)	5	+580	1,921	2860 → 3492

generation stationary—in five rounds for partitions 0 and 2–4 and nine for partition 1. *(ii) Refutation is dominated by the dataset; active search is a minor contributor.* Each round the datasets refute ∼245–360 candidate conjectures while the entire active-search ensemble (z3 / cross-entropy / VNS / SA / MCTS / RL) accounts for only 1–22. Per-round wall-time was ∼20–40 min, so a five-round partition finished in ≈2 h and the nine-round partition in ≈4.5 h.

Merging the five partitions' raw outputs (8,281 in total) and removing duplicate statements yields 7,935 survivors. The class-conditioned *inequalities* are disjoint across partitions (each partition owns distinct target invariants on the left-hand side), but the *Sophie* sufficient-conditions are not: their consequent or antecedent is a target-independent boolean property, so different partitions re-derive the same condition, and removing duplicates eliminates 346 such cross-partition repeats. The survivors split into 3,386 class-conditioned inequalities and 4,549 Sophie sufficient-conditions (no *unconditioned* inequality survives—disconnected graphs in the refutation dataset correctly eliminate bounds that tacitly assume connectivity, leaving only their class-conditioned forms). The novelty filter flags 33 survivors as rediscoveries of classical theorems—among them König–Egerváry $\nu \leq \tau$, $\nu \leq n/2$, Gallai $\tau = n - \alpha$ and $\tau \leq 2\nu$, Fajtlowicz–Saks/Chung rad $\leq \alpha$, $\gamma \leq i(G)$, and the perfect-graph identity $\alpha = \overline{\theta}$ recovered independently on the chordal, cograph, and bipartite classes—which is the intended correctness signal. The remaining "novel"-flagged survivors are dominated by *true-but-trivial* bounds the (inequality-only, table-limited) filter does not yet recognise—e.g. the trivial direction $\alpha \leq \overline{\theta}$—and by *definedness artefacts* such as the Sophie condition (radius $\leq \cdot$) ⇒ connected, whose hypothesis is only defined on connected graphs; both are exactly the failure modes of §4.3.

4.2. Rejecting the system's own apparent discoveries

The pipeline produced several apparent "discoveries." Every one was rejected by verification; we list them because the rejections, not the candidates, are what matters. The database artefacts $\delta \leq \lambda + 5$ and $\chi \leq 2m/n + 3$ held on the whole dataset but are false; the adversarial dataset exhibits the barbell and clique-plus-tail witnesses. The bound $n \leq m + 1$ is an instructive connectivity artefact: it merely asserts $m \geq n - 1$, so it holds on every connected graph and would "survive" on a connected-only dataset. In the HPC run we therefore deliberately include disconnected graphs in the refutation dataset (the $n \leq 9$ census dataset is taken in full, not connected-only), which refutes $n \leq m + 1$ immediately on any forest-plus-isolated-vertex witness. The same choice lets the *Sophie* sufficient-conditions be attacked properly: a condition such as (order ≤ 14) ⇒ connected can only be refuted by a disconnected witness, and is now caught rather than surviving unchallenged.

5. Discussion

Every search and exhaustive test we ran reached the same conclusion: within linear and product inequalities among classical invariants on standard graph collections, the survivors after filtering are classical theorems, trivial structural facts, or fitted/order artefacts of the database. We state this as a claim about the searched hypothesis class and our filters, not about the field as a whole: the generator explores (mostly linear and product) bounds over a classical invariant list, and the novelty filter is

curated from classical theorems, so the absence of surviving novelty partly reflects these design choices rather than the maturity of the field alone. With that scope acknowledged, the result is unsurprising, and the pipeline rediscovering Fajtlowicz–Saks (rad $\leq \alpha$) [44], Sumner–Las Vergnas ($n \leq 2\nu + 1$ for claw-free graphs) [35], and the greedy and domination product bounds is evidence that it functions correctly, rather than a shortcoming. Surveys of computer-assisted graph theory [18] and automated conjecture-making [44] confirm that this remains the typical experience in mature areas. Genuine novelty in this setting requires either leaving the inequality-fitting paradigm (structural/existential statements, program evolution for extremal constructions, neurosymbolic reasoning [45]) or matching exotic invariant definitions exactly against formal sources—both beyond inequality search over a finite database.

6. Conclusion

We described AutoGraphForge, an iterative conjecturing–refuting pipeline for graph theory with an autoformalization front-end. Its outcome is negative and, we argue, informative: over the linear, product, and expression-tree inequalities among classical invariants that the pipeline searches, the survivors after filtering are classical theorems, trivial structural facts, or fitting artefacts—the system functioning correctly. We stress that this is a statement about the searched hypothesis class and our inequality-only filter, not about graph theory as a whole. Our concrete contributions are an exact-or-blank 348,207-graph refutation dataset with a zero-mismatch cross-backend consistency gate, a validated structure-seeded counterexample engine (reproducing the Aouchiche–Hansen refutation to the published optimal score), exhaustive confirmation of several open conjectures on all connected graphs up to nine vertices, and a kernel-verifying Lean/DeepSeek-Prover back-end demonstrated on curated inequalities.

The main open tasks—and our current work—are closing the loop end to end: a class-conditioned novelty filter (the present implication filter operates on unconditioned bounds and does not yet recognise class-conditioned classics such as (chordal) $\Rightarrow \chi = \omega$ or (bipartite) $\Rightarrow \alpha = \overline{\theta}$); the present HPC run extends the earlier single pass to a multi-round, cluster-partitioned iteration budget (up to 25 rounds per partition, run to a fixed point or a wall-clock cap) whose survivors feed the scaled 671B proving pass; a *Devil's Advocate* filter that assumes each surviving conjecture true and feeds it back as a constraint on subsequent generation (the dashed path in Figure 1), and the end-to-end formal proof of a machine-generated survivor. Additionally, replacing the static generator with an LLM-guided evolutionary search is left for future system iterations; for the present version, the static generator (TxGraffiti2) was strictly preferred for its determinism, speed, and exactness.

Acknowledgments

I thank Samuel Varchol and Vladimír Jančár for valuable discussions on techniques of counterexample search.

This work was supported by the use of computational resources of the supercomputer PERUN, operated by the Supercomputing Centre at the Technical University of Košice (TUKE), Slovakia with the support of the European Union from the funds of the Recovery and Resilience Plan of the Slovak Republic within the framework of project No. 17I03-04-P03-00001, Development and design of a supercomputer for the National Supercomputing Center.

Declaration on Generative AI

During the preparation of this work, the author used a large language model as a coding and experimentation assistant to implement the pipeline, run the experiments, and help with the draft of the manuscript. All computational results were verified against authoritative data sources – House of Graphs and exact recomputation of GraphCalc; the author reviewed and edited all content and take full responsibility for the publication's content.

Code availability

All code and databases are available at https://github.com/JanPastorek/AutoGraphForge.

References

[1] L. de Moura, S. Kong, J. Avigad, F. Van Doorn, J. von Raumer, The Lean theorem prover (system description), in: International Conference on Automated Deduction, Springer, 2015, pp. 378–388.

[2] S. Fajtlowicz, On conjectures of Graffiti, Discrete Mathematics 72 (1988) 113–118.

[3] Wikipedia contributors, Graffiti (program) — Wikipedia, the free encyclopedia, https://en.wikipedia.org/wiki/Graffiti_(program), 2026.

[4] C. E. Larson, Intelligent machinery and mathematical discovery, Graph Theory Notes of New York XLII (2002) 8–17.

[5] E. DeLaVina, Graffiti.pc: A variant of Graffiti, Graph Theory Notes of New York XLII (2002) 26–30.

[6] E. DeLaVina, Some history of the development of Graffiti (2004). University of Houston.

[7] M. Aouchiche, P. Hansen, D. Stevanović, Variable neighborhood search for extremal graphs. 17. further conjectures and results about the index, Discussiones Mathematicae Graph Theory 29 (2009) 15–37.

[8] G. Caporossi, P. Hansen, Variable neighborhood search for extremal graphs. V. three ways to automate finding conjectures, Discrete Mathematics 276 (2004) 81–94.

[9] H. Mélot, Facet defining inequalities among graph invariants: The system GraPHedron, Discrete Applied Mathematics 156 (2008) 1875–1891.

[10] R. Davila, Automated conjecturing in mathematics with TxGraffiti, 2024. arXiv:2409.19379.

[11] Y. Caro, R. Davila, M. A. Henning, R. Pepper, Conjectures of TxGraffiti: Independence, domination, and matchings, Australasian Journal of Combinatorics 84 (2022) 258–274.

[12] R. Davila, The Optimist: Towards fully automated graph theory research, 2024. arXiv:2411.09158.

[13] C. E. Larson, N. V. Cleemput, Automated conjecturing I: Fajtlowicz's Dalmatian heuristic revisited, Artificial Intelligence 231 (2016) 17–38.

[14] C. E. Larson, N. V. Cleemput, Automated conjecturing I: Fajtlowicz's Dalmatian heuristic revisited (extended abstract), in: Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17), 2017.

[15] R. Davila, Graffiti^3: Compact theory libraries for automated mathematical discovery, Preprint, 2026.

[16] S. Bonte, G. Devillez, V. Dusollier, H. Mélot, PHOEG: an online tool for discovery and education in extremal graph theory, 2026. arXiv:2603.27242.

[17] T. L. Brewster, M. J. Dinneen, V. Faber, A computational attack on the conjectures of Graffiti: New counterexamples and proofs, Discrete Mathematics 147 (1995) 35–55.

[18] J. Jooken, Computer-assisted graph theory: a survey, 2025. arXiv:2508.20825.

[19] A. Z. Wagner, Constructions in combinatorics via neural networks, 2021. arXiv:2104.14516.

[20] Y. Wu, A. Q. Jiang, W. Li, M. N. Rabe, C. Staats, M. Jamnik, C. Szegedy, Autoformalization with large language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2022.

[21] D. D. Mavani, N. Pflueger, Formalizing chip-firing and riemann-roch for graphs in lean 4, arXiv preprint arXiv:2606.16679 (2026).

[22] N. Nader, I. Aljabea, P. Diehl, D. Gupta, Gtbench: A curriculum-grounded benchmark for evaluating llms as mathematical research assistants in graph theory, arXiv preprint arXiv:2606.03144 (2026).

[23] J. Kalfus, B. Lidický, An automated proof that r(b8, b10) = 37, arXiv preprint (2026).

[24] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, F. Liu, Leanmarathon: Toward reliable ai co-mathematicians through long-horizon lean autoformalization, arXiv preprint arXiv:2606.05400 (2026).

[25] DeepSeek-AI, Deepseek-prover-v2: Scalable moe transformer for formal theorem proving, DeepSeek Technical Report (2025).

[26] K. Coolsaet, S. D'hondt, J. Goedgebeur, House of graphs 2.0: a database of interesting graphs and more, Discrete Applied Mathematics 325 (2023) 97–107. doi:10.1016/j.dam.2022.10.013.
[27] J. Goedgebeur, An introduction to computational graph theory and generation algorithms, in: CEUR Workshop Proceedings (ITAT'22), 2022.
[28] R. Davila, GraphCalc: A Python package for computing graph invariants in automated conjecturing systems, Journal of Open Source Software 10 (2025) 8383. doi:10.21105/joss.08383.
[29] A. A. Hagberg, D. A. Schult, P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, in: Proceedings of the 7th Python in Science Conference (SciPy 2008), 2008, pp. 11–15.
[30] J. Christophe, S. Dewez, J.-P. Doignon, G. Fasbender, P. Grégoire, D. Huygens, M. Labbé, S. Elloumi, H. Mélot, H. Yaman, Linear inequalities among graph invariants: Using GraPHedron to uncover optimal relationships, Networks 52 (2008) 287–298.
[31] T. Gallai, über extreme punkt- und kantenmengen, Ann. Univ. Sci. Budapest, Eötvös Sect. Math. 2 (1959) 133–138.
[32] H. Whitney, Congruent graphs and the connectivity of graphs, American Journal of Mathematics 54 (1932) 150–168.
[33] D. Kőnig, Gráfok és mátrixok, Matematikai és Fizikai Lapok 38 (1931) 116–119.
[34] G. A. Dirac, Some theorems on abstract graphs, Proceedings of the London Mathematical Society s3-2 (1952) 69–81.
[35] D. P. Sumner, Graphs with 1-factors, Proceedings of the American Mathematical Society 42 (1974) 8–12.
[36] T. Cazenave, Nested Monte-Carlo search, in: Proc. 21st Int. Joint Conf. on Artificial Intelligence (IJCAI), 2009, pp. 456–461.
[37] C. D. Rosin, Nested rollout policy adaptation for Monte Carlo tree search, in: Proc. 22nd Int. Joint Conf. on Artificial Intelligence (IJCAI), 2011, pp. 649–654.
[38] L. de Moura, N. Bjørner, Z3: An efficient SMT solver, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008), volume 4963 of *Lecture Notes in Computer Science*, 2008, pp. 337–340. doi:10.1007/978-3-540-78800-3_24.
[39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023), 2023, pp. 611–626. doi:10.1145/3600006.3613165.
[40] M-A-P, OProver: A unified framework for agentic formal theorem proving, 2026. arXiv:2605.17283, model: https://huggingface.co/m-a-p/OProver-32B.
[41] F. van Doorn, G. Ebner, R. Y. Lewis, Maintaining a library of formal mathematics, in: Intelligent Computer Mathematics (CICM 2020), 2020. arXiv:2004.03673.
[42] M. Roucairol, T. Cazenave, Refutation of spectral graph theory conjectures with Monte Carlo search, in: LAMSADE, 2022.
[43] M. Roucairol, T. Cazenave, Refutation of spectral graph theory conjectures with search algorithms, in: IASE 2025, 2025.
[44] C. E. Larson, A survey of research in automated mathematical conjecture-making (????). Department of Mathematics, University of Houston.
[45] S. Feeney, P. Rao, A. Klappenecker, R. Tate, Y. Alexeev, S. Mensa, E. Kyoseva, S. Eidenbenz, SCALAR: A neurosymbolic framework for automated conjecture and reasoning in quantum circuit analysis, 2025.

A. Reproducibility

The methods are released as the AutoGraphForge package, built on TxGraffiti2 [10] and GraphCalc [28], with the following layout. The canonical invariant backend and the cross-backend consistency gate are `invariants/graphcalc_backend.py`, `invariants/networkx_backend.py`, and

`invariants/crossvalidate.py`; the exact-or-blank store and the TxGraffiti2 `KnowledgeTable` builder are `data/store.py` and `data/knowledge_table.py`, with `data/ingest.py` loading the combined dataset (HoG export and the exhaustive $n \leq 9$ census). Generation, the implication-deciding novelty filter, the Graffiti-lineage selection heuristics and IRIS scoring, and falsification over native TxGraffiti2 `Conjecture` objects are implemented under `pipeline/`: the iterative loop in `pipeline/cegis.py`, the implication-deciding novelty filter in `pipeline/novelty.py` and `pipeline/cegis_novelty.py`, the adversarial refutation dataset in `pipeline/adversarial.py`, `pipeline/falsification.py` and `pipeline/refute_matrix.py`, and the interchangeable counterexample-search backends in `pipeline/search/` (`metaheuristics.py` for VNS/SA/MCTS, `z3_adapter.py` for the SMT backend, and `rlgt_adapter.py` for the linear and deep cross-entropy refuters); found witnesses are re-inserted into the database to tighten subsequent rounds (the dynamic database). Counterexample *minimisation* in the Graffiti tradition (the Red Burton rule) is not yet implemented and is listed as future work (§6). The Larson–Van Cleemput Conjecturing engine is run as a separate, optional experiment through the Sage drivers under `pipeline/conjecturing/` across a file-spec boundary; the coefficient-tuning and expression-bridge prototypes are kept in `legacy/` and are not part of the pipeline evaluated here. The stages are split across independent workers (`tools/run_shard.py` partitions the invariant list; `tools/merge_shards.py` reconciles the per-partition survivors and hard-witness seeds), with the SLURM batch scripts for the partitioned multi-round array and the vLLM-served 671B proving stage provided under `tools/slurm/`; the same subcommands (`autographforge ingest` | `crossvalidate` | `discover` | `conjecture`) run on a single machine and, under a scheduler, on a cluster. The consistency gate (0 mismatches over the shared exact invariants) and the loading path are covered by the package test suite.